

Indian Telecom Churn & A/B Testing Project

Learning & Interview Preparation Guide

Covers: Phase 0 (Setup) and Phase 1 (Exploratory Data Analysis)

Table of Contents

Table of Contents.....	2
Introduction: How to Use This Guide.....	4
Phase 0: Project Setup, Git & GitHub.....	5
0.1 What is Python, and why use it for data analysis?	5
0.2 What is a virtual environment, and why did we use 'venv312'?	5
0.3 What is Jupyter Notebook, and why use it for EDA?	5
0.4 What is Git, and what problem does it solve?	6
0.5 What is GitHub, and how is it different from Git?.....	6
0.6 Understanding the merge conflict you encountered	6
0.7 Project folder structure - and why it's organized this way	7
0.8 What is .gitignore, and why do we need it?	7
Phase 1: Exploratory Data Analysis (EDA).....	9
1.1 pandas DataFrame basics.....	9
df.shape.....	9
df.info()	9
df.describe().....	9
df.isnull().sum()	9
1.2 Univariate Analysis - examining one variable at a time	11
Why visualize distributions instead of just reading numbers?.....	11
Bar chart vs Histogram - what's the difference?	11
What is 'KDE' (the smooth curve on histograms)?.....	11
Reading the operator distribution	11
1.3 Bivariate Analysis - relationships between two variables.....	13
Understanding groupby() - the core tool	13
Project Finding 1: Churn rate by Operator	13
Project Finding 2: Churn rate by Circle Type (Metro/Urban/Rural)	13
Project Finding 3: Churn rate by Number of Complaints	14
Project Finding 4: Churn rate by VLR (Active Subscriber) Status - the strongest signal	14
Project Finding 5: Churn rate by Recharge Recency (with binning).....	14
1.4 Correlation Analysis	16
What is correlation, from scratch?	16
How df.corr() works	16
Our results: correlation with churn, sorted	16
CRITICAL CONCEPT: Why correlation can UNDERSTATE a real relationship	17
Why we're saving deeper modeling for Phase 3.....	17

Project Context: Grounding in Real TRAI Data	19
How to Tell the Story of This Project in an Interview	19
Quick-Reference Cheat Sheet	20
pandas methods used	20
Visualization methods used.....	20
Statistical concepts covered.....	20
Git/GitHub commands used	21
What's Next	21

Introduction: How to Use This Guide

This guide teaches every concept used so far in your Indian Telecom Churn & A/B Testing project, from first principles. It is written for someone with little to no prior background in statistics, data analysis, or Git/GitHub.

Each section follows the same pattern:

- Concept explained in plain English (the 'what' and 'why')
- How it applies specifically in YOUR project
- Interview angle: how this topic is likely to be asked, and a strong sample answer

Use this as a companion document while you work through the project, and as direct revision material before data analyst / data scientist interviews. Everything here maps to files and commits already in your repository.

Phase 0: Project Setup, Git & GitHub

0.1 What is Python, and why use it for data analysis?

Python is a general-purpose programming language that has become the most popular language for data analysis and machine learning because of its readable syntax and a huge ecosystem of free libraries built specifically for working with data:

- pandas - for loading, cleaning, and manipulating tabular data (like Excel, but programmable)
- numpy - for fast numerical operations on arrays of numbers
- matplotlib / seaborn - for creating charts and visualizations
- scikit-learn - for machine learning models
- statsmodels / scipy - for statistical tests (used heavily in A/B testing)

Anaconda is a distribution of Python that comes pre-packaged with most of these libraries, plus Jupyter Notebook, so beginners don't have to install everything manually.

0.2 What is a virtual environment, and why did we use 'venv312'?

A virtual environment is an isolated, self-contained copy of Python and its installed packages, separate from your computer's main Python installation. Think of it as a separate 'workspace' for one project.

Why this matters:

- Different projects may need different versions of the same library (e.g. Project A needs pandas 1.5, Project B needs pandas 2.1). Without virtual environments, installing one would break the other.
- It keeps your project's dependencies documented and reproducible - anyone else (or you, on a new machine) can recreate the exact same setup.
- 'venv312' in your terminal prompt means you're inside a virtual environment named 'venv312' (likely created with Python 3.12).

Basic commands (Windows):

```
python -m venv venv312          # create the environment
venv312\Scripts\activate        # activate it (you'll see (venv312) in prompt)
pip install pandas numpy matplotlib seaborn jupyter  # install packages inside it
deactivate                      # exit the environment
```

Q: Why do data scientists use virtual environments?

A: Virtual environments isolate project-specific package versions so dependencies don't conflict across projects, and they make a project's setup reproducible - anyone can recreate the exact environment using a requirements.txt file. This is considered a best practice for production-quality and collaborative data science work.

0.3 What is Jupyter Notebook, and why use it for EDA?

Jupyter Notebook is an interactive coding environment where you write and run code in small chunks called 'cells'. Each cell can be run independently, and its output (tables, charts, printed text) appears directly below it, mixed with your code.

This is ideal for exploratory work because:

- You can inspect data step-by-step rather than running an entire script at once
- Charts render inline, right next to the code that created them
- You can add markdown text cells to document your thinking, narrate findings, and explain decisions - which is exactly how we wrote up the EDA findings in Phase 1

The trade-off: notebooks are great for exploration but not ideal for production code (which is why, later in the project, we'll move reusable logic into .py scripts under src/).

0.4 What is Git, and what problem does it solve?

Git is a version control system - software that tracks changes to your files over time. Every time you 'commit', Git saves a snapshot of your project at that moment, with a message describing what changed.

Why this matters for a portfolio project:

- You can see (and show recruiters) the evolution of your project - how you approached the problem step by step
- You can undo mistakes by going back to a previous snapshot
- Multiple people can work on the same project without overwriting each other's work

Key Git concepts used so far:

git init: initializes a new Git repository in your project folder - starts tracking changes.

git add: stages files - tells Git 'include these changes in the next snapshot'.

git commit -m "message": creates a snapshot (a 'commit') of all staged changes, with a descriptive message.

git push: uploads your local commits to a remote repository (GitHub, in our case).

git pull: downloads commits from the remote repository that you don't have locally.

0.5 What is GitHub, and how is it different from Git?

Git is the tool that runs on your computer and tracks history. GitHub is a website that hosts Git repositories online, so your code is backed up, shareable, and visible to others (like recruiters).

A 'remote' is a reference to a repository hosted elsewhere (e.g. on GitHub). 'origin' is just the conventional name Git gives to the main remote.

0.6 Understanding the merge conflict you encountered

When you ran 'git push' and got a 'rejected... fetch first' error, here is what was actually happening:

1. You created a repository on GitHub and initialized it with a LICENSE file (and possibly other files). This created one commit on GitHub that exists only on the remote.
2. Separately, on your computer, you ran 'git init' and made your own first commit ('Initial project structure...'). This commit exists only locally.
3. These two commits have no shared history - Git calls this 'unrelated histories'.
4. When you tried to push, Git refused because pushing would have overwritten the GitHub commit you didn't have locally - Git protects you from silently losing work.

The fix - 'git pull origin main --allow-unrelated-histories' - downloaded the GitHub commit, and Git automatically merged it with your local commit into a new 'merge commit' (you saw this as commit 9bf3ded). Since the GitHub repo only had a LICENSE (and your local repo didn't touch that file), there was no actual conflict to resolve - Git merged them automatically.

'--allow-unrelated-histories' is needed because, by default, Git assumes two completely separate commit histories merging together is probably a mistake (e.g. merging two different projects), so it requires you to explicitly confirm this is intentional.

Q: What is a merge conflict, and how do you resolve one?

A: A merge conflict happens when Git can't automatically combine changes because the same part of a file was modified differently in two histories being merged. Git marks the conflicting sections in the file with <<<<<<, =====, and >>>>>> markers. To resolve it, you manually edit the file to keep the correct content, remove the markers, then 'git add' the resolved file and 'git commit' to complete the merge. In my case, the conflict was actually an 'unrelated histories' situation (no overlapping file changes), so Git merged it automatically once I allowed unrelated histories.

0.7 Project folder structure - and why it's organized this way

Our repository structure:

```
indian-telecom-churn-analysis/
├── data/
│   └── raw/                # original, untouched datasets
├── notebooks/             # Jupyter notebooks for exploration
├── src/                   # reusable Python scripts/modules
├── reports/               # write-ups, figures, exported results
├── .gitignore
├── LICENSE
└── README.md
```

This structure follows a common data science project convention. The reasoning:

- Separating 'raw' data signals that this data should never be edited directly - any cleaning happens in code, so the original is always reproducible
- 'notebooks' vs 'src' separates exploration (messy, iterative) from production-quality reusable code (clean functions/classes)
- This structure is recognizable to any data professional skimming your repo - it signals you understand how real data teams organize work

0.8 What is .gitignore, and why do we need it?

A .gitignore file tells Git which files or folders to NEVER track or upload. We excluded:

.ipynb_checkpoints/: auto-save backup folders Jupyter creates - clutter, not needed in version control

__pycache__/: compiled Python bytecode files - automatically regenerated, machine-specific

venv/: the virtual environment folder - it can be huge (hundreds of MB) and is fully reproducible from a requirements.txt file, so committing it would bloat the repo unnecessarily

***.pyc**: compiled Python files

.env: files that typically store secrets/passwords/API keys - NEVER commit these, even accidentally, as they can leak credentials

Q: Why shouldn't you commit your virtual environment or .env files to GitHub?

A: Virtual environments are large, machine-specific, and fully reproducible from a requirements.txt or environment.yml file, so committing them bloats the repository for no benefit. .env files often contain secrets like API keys or database passwords - if committed, especially to a public repo, those credentials become exposed to anyone, including automated bots that scan GitHub for leaked secrets. The standard practice is to commit a '.env.example' template without real values, and add '.env' to .gitignore.

Phase 1: Exploratory Data Analysis (EDA)

EDA is the process of looking at a dataset before doing any modeling, to understand its structure, quality, and the patterns it contains. The goal is to build intuition about the data and form hypotheses that modeling will later test. Skipping EDA is one of the most common mistakes - it leads to building models on data you don't actually understand, which produces unreliable or misleading results.

1.1 pandas DataFrame basics

A DataFrame is pandas' core data structure - think of it as a table, like a spreadsheet, where each column can hold a different type of data (numbers, text, dates), but every column has the same number of rows.

```
import pandas as pd
df = pd.read_csv('../data/raw/indian_telecom_churn.csv')
```

'pd.read_csv()' reads a CSV (Comma-Separated Values) file - a plain-text format where each line is a row and commas separate column values - and loads it into a DataFrame. 'df' is just a variable name (short for 'dataframe'), a convention almost everyone uses.

df.shape

Returns a tuple (rows, columns) - in our case (10000, 20). This is the very first thing to check: it tells you the size of your dataset and whether it loaded correctly (e.g. if you expected 10,000 rows but got 1, something went wrong with the file path or delimiter).

df.info()

Shows, for each column: its name, how many non-null (non-missing) values it has, and its data type (dtype). This is your dataset's 'health check'. Key things to look for:

- Do the dtypes make sense? (e.g. a column of numbers showing as 'object'/text often means there's a stray non-numeric value, like 'N/A' written as text, somewhere in that column)
- Does 'Non-Null Count' match the total row count for every column? If not, you have missing values to handle

In our dataset, all 20 columns show 10000 non-null - meaning zero missing values. Numeric columns (age, plan_amount_inr, tenure_months, etc.) are int64 or float64, and text columns (customer_id, name, operator, etc.) show as 'str' (object) - both as expected.

df.describe()

Generates summary statistics for numeric columns by default: count, mean, std (standard deviation), min, 25%/50%/75% percentiles, and max. Passing include='all' also summarizes text/categorical columns (unique count, most frequent value, its frequency).

Why this matters: describe() is a fast way to spot outliers or data errors. For example, if 'age' showed a max of 250 or a min of -5, that would signal a data quality problem before you've even plotted anything.

df.isnull().sum()

'isnull()' returns a same-shaped DataFrame of True/False values (True = missing). '.sum()' then counts the Trues per column (Python treats True as 1, False as 0 when summed). Our output showed 0 for every column - a clean dataset with no missing values, which is realistic here

since we generated the data synthetically, but real-world datasets almost always have missing values that need handling (a topic for Phase 2).

Q: Walk me through your first steps when you receive a new dataset.

A: First I check `df.shape` to confirm the size matches expectations. Then `df.info()` to check data types and spot any columns with unexpected types or missing values. Then `df.describe()` to look at the distribution of numeric columns and catch obvious data quality issues like impossible values or extreme outliers. Then `df.isnull().sum()` to quantify missing data precisely, since `info()` only gives non-null counts. Only after this 'health check' do I move into deeper univariate and bivariate analysis - this order prevents wasting time analyzing patterns in data that turns out to have quality issues.

Q: What would you do if you found missing values in a real dataset?

A: It depends on the amount and the mechanism. First I'd check what percentage of rows are affected and whether missingness is random or correlated with other variables (e.g. maybe 'income' is missing more often for self-employed customers - that's not random, and dropping those rows would bias the analysis). Options include: dropping rows/columns if missingness is small and random; imputing with mean/median/mode for simple cases; using more sophisticated imputation (e.g. predictive imputation) for important features; or creating a 'missing' flag/category if the missingness itself might be informative (e.g. customers who didn't report income might behave differently).

1.2 Univariate Analysis - examining one variable at a time

Univariate analysis means looking at the distribution of a single column in isolation - what values does it take, how often, and what shape does that distribution have? This is the foundation before looking at relationships between variables.

Why visualize distributions instead of just reading numbers?

A chart reveals shape information that summary statistics can hide. For example, two columns could have the same mean and standard deviation but completely different shapes - one could be a normal 'bell curve', the other could have two separate peaks (bimodal). Visualizing catches this immediately.

Bar chart vs Histogram - what's the difference?

Bar chart (`sns.countplot`): used for categorical data - each bar represents a distinct category (e.g. 'Jio', 'Airtel', 'Vi', 'BSNL'), and the bar's height shows the count of rows in that category.

Histogram (`sns.histplot`): used for continuous numeric data - the x-axis range is split into 'bins' (intervals), and each bar shows how many values fall into that interval. This approximates the shape of the underlying distribution.

In our project: 'churn', 'operator', 'circle_type', 'plan_type', 'payment_method', and 'gender' are categorical -> bar charts. 'age', 'tenure_months', 'plan_amount_inr', 'data_usage_gb', 'days_since_last_recharge' are continuous -> histograms.

What is 'KDE' (the smooth curve on histograms)?

KDE stands for Kernel Density Estimate. It's a smoothed version of a histogram - instead of discrete bars, it draws a continuous curve that estimates the underlying probability density of the data. We added `kde=True` to the `tenure_months` histogram to see its overall shape more clearly (in our case, a right-skewed/exponential-like shape - many customers with short tenure, fewer with long tenure, tapering off gradually).

Reading the operator distribution

Our operator countplot, ordered by `value_counts()`, showed Jio (~39.8%) and Airtel (~37.5%) as the largest groups, with Vi (~15.2%) and BSNL (~7.4%) smaller. This mirrors the real-world TRAI market share data we used to calibrate the synthetic dataset - a key point for your write-up, since it shows your synthetic data is grounded in real industry context rather than arbitrary.

Q: When would you use a histogram vs a bar chart?

A: A histogram is for continuous numeric variables, where the x-axis is divided into intervals (bins) and bar height shows frequency within each interval - it shows the shape of a distribution, like skewness or whether it's normal/bimodal. A bar chart is for categorical variables, where each bar represents a distinct, separate category with no inherent numeric ordering between bars (though they can be ordered by frequency for readability). Using a bar chart for continuous data, or a histogram for categories, would misrepresent the data - histograms imply a continuous numeric scale on the x-axis.

Q: What does it mean for a distribution to be 'right-skewed', and why does it matter?

A: A right-skewed (positively skewed) distribution has a long tail extending toward higher values - most values are clustered on the lower end, with a few much larger values pulling the tail to the right. Our `tenure_months` distribution is right-skewed: most customers are relatively new, with a smaller number of long-tenured customers. This matters because: (1) the mean gets pulled toward the tail and becomes less representative than the median; (2) some models (like

linear regression) assume roughly normal distributions and may benefit from a log transformation of skewed features; (3) it can reveal business realities - here it reflects natural customer turnover, where most of the base is relatively recent.

1.3 Bivariate Analysis - relationships between two variables

Bivariate analysis examines how two variables relate to each other - in our project, specifically how each feature relates to 'churn' (our target/outcome variable). This is where we start forming hypotheses about what drives churn.

Understanding groupby() - the core tool

```
df.groupby('operator')['churn'].mean()
```

Read this in three steps:

5. `groupby('operator')`: splits the DataFrame into separate groups, one per unique value of 'operator' (Jio, Airtel, Vi, BSNL)
6. `['churn']`: within each group, select only the 'churn' column
7. `.mean()`: calculate the average of 'churn' within each group

Since 'churn' is encoded as 0 (stayed) or 1 (churned), the MEAN of a column of 0s and 1s equals the PROPORTION of 1s - i.e. the churn rate. This is a very common and useful trick: averaging a binary (0/1) column directly gives you a rate or percentage.

Q: If a column contains only 0s and 1s, what does its mean represent?

A: The proportion (or rate) of 1s in that column. For example, if 'churn' is 1 for churned customers and 0 otherwise, then `df['churn'].mean()` equals the overall churn rate - because the sum of the column (total number of 1s) divided by the count (total rows) is exactly the proportion. This is why `groupby(...)[ 'binary_column' ].mean()` is such a common pattern for computing rates by category.

Project Finding 1: Churn rate by Operator

BSNL	0.221477
Vi	0.212459
Airtel	0.071391
Jio	0.065644

Interpretation: BSNL and Vi customers churn at roughly 21-22%, while Jio and Airtel customers churn at roughly 6.5-7%, a difference of about 3x. This was deliberately built into the synthetic data, calibrated to TRAI's real reporting that Vi and BSNL have substantially lower 'active subscriber' (VLR) ratios than Jio and Airtel - i.e. a larger share of their registered subscribers are not actually using their service, which correlates with eventual churn/porting out.

Business interpretation: if you were presenting this to a telecom company, this finding alone tells a retention team WHERE to focus first - Vi and BSNL's subscriber bases carry much higher churn risk, so retention budget should be weighted accordingly.

Project Finding 2: Churn rate by Circle Type (Metro/Urban/Rural)

Urban	0.106721
Rural	0.097835
Metro	0.096912

Interpretation: the differences here (9.7% to 10.7%) are small - roughly a 1 percentage point spread. Compare this to the operator finding (15 percentage point spread). This tells us circle type, on its own, is NOT a strong driver of churn in this dataset. An important EDA skill is recognizing not just strong patterns, but ALSO weak ones - reporting 'this variable doesn't seem to matter much' is just as valuable as reporting one that does, because it focuses later modeling effort appropriately.

Project Finding 3: Churn rate by Number of Complaints

```
0    0.093165
1    0.093541
2    0.170784
3    0.144330
4    0.187500
5    0.000000 <- only 1 customer in this group!
```

Interpretation: there's a general upward trend - 0-1 complaints sit around 9%, while 2-4 complaints sit around 14-19%. But the '5 complaints -> 0% churn' row is misleading, because it's based on a SINGLE customer (see `value_counts()` below). This is a critical lesson in EDA.

```
0    5560
1    3282
2     931
3     194
4      32
5       1 <- sample size of 1
```

Q: Why is it dangerous to draw conclusions from small subgroups in your data?

A: Because rates calculated from small sample sizes are highly unstable and can be driven entirely by chance - with a sample of 1, the 'churn rate' can only be exactly 0% or 100%, neither of which tells you anything reliable about the broader population of customers with that characteristic. This is related to the statistical concept of variance/standard error being inversely related to sample size - smaller samples produce noisier estimates. In practice, I'd either group small categories together (e.g. '4+ complaints'), report confidence intervals alongside rates, or explicitly flag low-n groups as unreliable rather than treating them the same as groups with thousands of observations.

Project Finding 4: Churn rate by VLR (Active Subscriber) Status - the strongest signal

```
is_active_vlr
0    0.317411 <- inactive customers
1    0.086836 <- active customers
```

Interpretation: this is by far the strongest bivariate relationship found. Customers flagged as 'inactive' (not appearing in the operator's VLR - Visitor Location Register, essentially a record of currently-connected devices) churn at 31.7%, nearly 4x the rate of active customers (8.7%). VLR status acts almost as an early-warning signal: a customer who has effectively 'gone dark' on the network is very likely to formally churn soon after.

VLR (Visitor Location Register) explained simply: it's a database telecom operators maintain of which subscriber devices are currently registered and reachable on the network. If a customer's SIM hasn't connected to any tower recently, they won't appear in the VLR - flagging them as 'inactive'. TRAI uses VLR-based active subscriber counts as a more meaningful metric than raw subscriber counts, since raw counts include many dormant/unused SIMs.

Project Finding 5: Churn rate by Recharge Recency (with binning)

We used `pd.cut()` to convert the continuous variable 'days_since_last_recharge' into categorical buckets (0-7, 8-15, 16-30, 31-45, 45+ days).

```
pd.cut(df['days_since_last_recharge'],
       bins=[-1, 7, 15, 30, 45, 200],
       labels=['0-7', '8-15', '16-30', '31-45', '45+'])
```

Q: Explain what `pd.cut()` does and why you might bin a continuous variable.

A: `pd.cut()` takes a continuous numeric column and a set of bin edges, and assigns each value to a labeled interval (bucket/bin) based on which range it falls into. The bins argument `[-1, 7, 15, 30, 45, 200]` creates 5 intervals: `(-1,7]`, `(7,15]`, `(15,30]`, `(30,45]`, `(45,200]` - and the labels argument gives each a readable name. Reasons to bin a continuous variable: (1) it makes groupby summaries and bar charts more readable than many individual values; (2) it can reveal threshold effects - non-linear relationships where the effect only appears past a certain point, which a single correlation coefficient would miss; (3) for some models, or for stakeholder communication, categorical 'risk tiers' are more actionable than a raw number.

0-7	0.084848
8-15	0.091781
16-30	0.091767
31-45	0.094228
45+	0.212000

Interpretation: this is a textbook example of a THRESHOLD EFFECT. Churn rate is essentially flat (8.5-9.4%) across 0-45 days, then jumps sharply to 21.2% beyond 45 days. A linear correlation coefficient would understate this relationship, because most of the 'action' happens in one sub-range of the variable, not gradually across its whole range. This is exactly why binning plus groupby is a valuable complement to correlation analysis - it can surface effects that a single number (correlation) hides.

Business interpretation: '45 days since last recharge' looks like a natural operational threshold for triggering a re-engagement campaign, before the customer crosses into much higher churn risk.

1.4 Correlation Analysis

What is correlation, from scratch?

Correlation measures the strength and direction of a LINEAR relationship between two numeric variables. The most common measure is the Pearson correlation coefficient, usually written as 'r', which ranges from -1 to +1:

r = +1: perfect positive linear relationship - as one variable increases, the other increases proportionally

r = -1: perfect negative linear relationship - as one variable increases, the other decreases proportionally

r = 0: no linear relationship (but there could still be a non-linear one - see below)

In practice, real-world correlations are rarely close to -1 or +1. As rough guidance: $|r|$ above 0.5 is often considered a strong relationship in social/business data, 0.3-0.5 moderate, 0.1-0.3 weak, and below 0.1 negligible - though these thresholds depend heavily on context and field.

How df.corr() works

```
corr = df[numeric_cols].corr()
```

This computes the Pearson correlation between every pair of columns in `numeric_cols`, producing a square matrix (a table where rows and columns are the same variables). The diagonal is always 1.0 (every variable is perfectly correlated with itself). The matrix is symmetric - `corr(A,B)` equals `corr(B,A)`.

A heatmap (`sns.heatmap`) visualizes this matrix with color: typically red/warm for positive correlation, blue/cool for negative, with intensity showing strength. The `'annot=True'` parameter overlays the actual numeric values on each cell.

Our results: correlation with churn, sorted

churn	1.000000
days_since_last_recharge	0.098479
num_complaints_6m	0.059011
age	0.012583
uses_5g	0.003410
data_usage_gb	-0.010405
plan_amount_inr	-0.036305
has_bundle	-0.064452
network_rating	-0.070787
tenure_months	-0.072599
is_active_vlr	-0.187847

Interpretation:

- `is_active_vlr` (-0.188) is the strongest individual predictor - the negative sign means HIGHER values of `is_active_vlr` (i.e. =1, active) are associated with LOWER churn, which matches Finding 4.
- `days_since_last_recharge` (+0.098) and `num_complaints_6m` (+0.059) are positively correlated - more days since recharge and more complaints associate with more churn, matching Findings 3 and 5.
- `tenure_months` (-0.073) and `network_rating` (-0.071) are weakly negative - longer tenure and better network ratings slightly associate with less churn.

- Everything else (age, uses_5g, data_usage_gb, plan_amount_inr) has $|r|$ below 0.04 - essentially no linear relationship with churn on their own.

CRITICAL CONCEPT: Why correlation can UNDERSTATE a real relationship

Notice that days_since_last_recharge has a correlation of only 0.098 with churn - this sounds weak. But our binned analysis (Finding 5) showed a dramatic jump from about 9% to about 21% churn after the 45-day threshold. How can both be true?

The answer: Pearson correlation only captures LINEAR relationships - it essentially asks 'as X goes up by one unit, does Y go up or down by a roughly constant amount, on average, across the whole range?' But our relationship is NOT linear - it's FLAT for most of the range (0-45 days) and then jumps. A linear correlation coefficient, trying to fit ONE straight line across the whole range, ends up with a low slope because most of the data shows no relationship, diluting the signal from the 45-plus day group.

This is one of the most important takeaways from this phase: correlation coefficients are a useful FIRST PASS, but they can hide threshold effects, non-linear relationships, and interactions between variables. This is exactly why we did BOTH binned groupby analysis AND correlation analysis - they catch different things. It's also a strong argument for why tree-based models (Random Forest, XGBoost - Phase 3) often outperform linear models on this kind of data: trees can naturally learn threshold/split-point effects like 'if days_since_last_recharge is greater than 45, risk increases sharply'.

Q: A variable shows a low correlation with your target, but your groupby analysis showed a clear pattern. How do you explain this discrepancy?

A: This usually means the relationship is non-linear - for example, a threshold effect where the variable only matters past a certain value, or a non-monotonic relationship. Pearson correlation specifically measures linear association, fitting a single straight-line relationship across the entire range of the variable. If most of the range shows no relationship and only a small sub-range shows a strong effect, that effect gets diluted in the overall correlation coefficient. This is why I always pair correlation analysis with binned groupby summaries and visualizations - they can reveal threshold or non-linear effects that a single summary statistic would mask. It's also a good argument for using models that can capture non-linearities, like tree-based methods, rather than relying purely on linear models guided by correlation.

Q: What's the difference between correlation and causation? Give an example from your project.

A: Correlation means two variables move together statistically; causation means one variable's change directly produces a change in the other. Correlation alone cannot establish causation, because of potential confounding variables - a third factor influencing both. In my project, is_active_vlr (active subscriber status) is strongly correlated with churn, but it doesn't necessarily CAUSE churn directly - both might be driven by an underlying factor, like general customer dissatisfaction or a competitor's better offer, which causes a customer to both stop using the SIM (going VLR-inactive) and eventually formally port out (churn). This is exactly why, in Phase 4 (A/B testing), we don't just rely on observed correlations - we run a randomized experiment, which is the gold-standard method for establishing causation, because random assignment breaks the link between the treatment and any confounding variables.

Why we're saving deeper modeling for Phase 3

EDA's job is to build understanding and generate hypotheses, not to draw final conclusions. The patterns found here (operator, VLR status, recharge recency, complaints) become the FEATURES we'll feed into predictive models in Phase 3, where we can quantify their combined,

and possibly interactive, effects on churn more rigorously - and validate which patterns hold up when controlling for other variables simultaneously, something a single groupby can't do.

Project Context: Grounding in Real TRAI Data

A common weakness in portfolio projects is using a synthetic or generic dataset with no real-world grounding. We addressed this by calibrating our synthetic data to real statistics published by the Telecom Regulatory Authority of India (TRAI), as of December 2025:

- Wireless market share: Airtel ~37.24%, Jio ~39.31%, Vi ~15.98%, BSNL ~7.46% - used to set the proportion of customers assigned to each operator in our synthetic dataset.
- Total wireless subscribers: approximately 1,244 million, with tele-density (subscribers per 100 people) of 88.41%.
- Monthly MNP (Mobile Number Portability) requests: approximately 16.12 million - this represents the real-world monthly 'churn' signal (customers switching operators while keeping their number).
- Vi's active subscriber (VLR) ratio (~85%) and BSNL's (~58-61%) are notably lower than Jio/Airtel's (~98%) - this real pattern directly informed our decision to make Vi and BSNL higher-churn-risk operators in the synthetic data, and to make `is_active_vlr` a strong churn predictor.

Q: Why did you build a synthetic dataset instead of using a real one from Kaggle?

A: Two reasons. First, the most common churn dataset (Telco Customer Churn) is extremely overused in portfolios, so it doesn't differentiate a candidate. Second, building my own dataset let me demonstrate additional skills: I researched real TRAI telecom statistics, calibrated a synthetic data generator to match real-world market share and churn patterns, and can explain and defend every assumption in the data because I made them deliberately. This also let me design my own A/B test scenario on top of the same customer base, which a pre-made dataset wouldn't support as cleanly.

Q: What are the limitations of using synthetic data, and how would you address them in a real role?

A: Synthetic data reflects the assumptions and relationships I built into the generator - it can't reveal genuinely unexpected patterns the way real data can, and it doesn't capture the messiness (missing values, data entry errors, inconsistent formats) of real operational data. In a real role, I'd validate any model built on synthetic data against real data before deployment, and I'd expect to spend significant time on data cleaning and handling real-world data quality issues that this synthetic dataset doesn't have. The value of this project is in demonstrating the analytical workflow, statistical reasoning, and communication - not in claiming the specific numbers represent real Indian telecom customers.

How to Tell the Story of This Project in an Interview

Interviewers often ask 'walk me through a project from your portfolio.' Here's a structure for Phase 0-1, which you'll extend as later phases complete:

8. Start with the business problem: 'Customer churn is a major cost driver for telecom operators. I wanted to build an end-to-end analysis - from raw data to a model to an experiment - mirroring how a real data team would approach a retention problem.'
9. Mention the data grounding: 'I researched real TRAI subscriber statistics to calibrate a synthetic dataset, so the patterns reflect real market dynamics - for example, Vi and BSNL have lower active-subscriber ratios in reality, so I built higher churn risk into those segments.'

10. Walk through your EDA process: 'I started with a data quality check - no missing values, correct types. Then univariate analysis to understand each variable's distribution, then bivariate analysis against churn.'
11. Highlight your STRONGEST finding with numbers: 'The clearest signal was active subscriber status - inactive customers churned at 31.7% versus 8.7% for active ones, a roughly 4x difference.'
12. Show analytical maturity by mentioning a NUANCE: 'I also found a threshold effect - churn was flat until 45 days since last recharge, then jumped sharply. This wouldn't show up strongly in a simple correlation, which is why I used both correlation analysis AND binned group comparisons.'
13. Mention what you'd do next: 'These findings fed directly into feature selection for the churn prediction models in Phase 3, and the high-risk segments became the target population for the retention A/B test in Phase 4.'

Quick-Reference Cheat Sheet

pandas methods used

pd.read_csv(path): load a CSV file into a DataFrame

df.shape: (rows, columns) tuple

df.info(): column names, dtypes, non-null counts

df.describe(include='all'): summary statistics for all columns

df.isnull().sum(): count of missing values per column

df['col'].value_counts(normalize=True): frequency (or proportion, if normalize=True) of each unique value

df.groupby('col')['target'].mean(): average of 'target' within each group of 'col' - gives a rate if target is 0/1

pd.cut(series, bins=[...], labels=[...]): convert a continuous variable into labeled categorical bins

df[cols].corr(): Pearson correlation matrix between numeric columns

Visualization methods used

sns.countplot(data=df, x='col'): bar chart of category frequencies

sns.histplot(data['col'], bins=N, kde=True): histogram with optional smoothed density curve

df[cols].hist(figsize=..., bins=N): grid of histograms, one per numeric column

sns.heatmap(corr, annot=True, cmap='coolwarm', center=0): color-coded correlation matrix with numeric labels

Statistical concepts covered

Mean of a binary column = rate/proportion: core trick for computing churn rates via groupby

Pearson correlation (r, range -1 to +1): measures linear association between two numeric variables

Correlation vs causation: correlation does not imply one variable causes the other - confounders may explain both

Threshold / non-linear effects: relationships that only appear past a certain value; not captured well by correlation

Small sample size instability: rates from very small groups (e.g. $n=1$) are unreliable and shouldn't be over-interpreted

Skewness: asymmetric distributions (e.g. right-skewed = long tail toward higher values)

Git/GitHub commands used

git init: start tracking a folder with Git

git add .: stage all changed files

git commit -m "message": save a snapshot with a description

git push -u origin main: upload commits to GitHub, set tracking branch

git pull origin main --allow-unrelated-histories: merge a remote history that shares no common commits with your local history

What's Next

Phase 2 (Preprocessing) will introduce: encoding categorical variables (one-hot vs label encoding), feature engineering, train/test splits, and handling class imbalance - each of these will get the same from-scratch treatment plus interview Q&A in a follow-up guide. Keep this document alongside your repo and revisit the Q&A sections before interviews.